Instituto Tecnológico de Costa Rica

Escuela Ingeniería en Computadores

Curso: CE1104 Fundamentos de Sistemas Computacionales

Documentación técnica StrangerTEC

Estudiantes:

Cristopher Viquez

David Ramirez

Profesor: Leonardo Araya

Semestre I 2026

0. Indice

0. Indice **2**
1. Introducción **4**
1.1. Propósito del documento 4
1.2. Descripción del proyecto 4
1.3 Objetivos y Alcance 4
1.4 Definiciones, Acrónimos y Abreviaturas 4
2. Descripción General del Sistema **5**
2.1. Perspectiva del Producto 5
2.2. Funcionalidades principales 5
2.3. Restricciones del sistema, Supuestos y Dependencias 5
3. Requisitos del Sistema **7**
3.1. Requisitos Funcionales 7
3.2. Requisitos No Funcionales 7
4. Diseño de la Solución **8**
4.1. Visión General de la Arquitectura 8
4.2. Descripción de Componentes 8
4.4. Diagrama de Flujo y Secuencia 9
4.5. Decisiones de Diseño y Justificación 9
5. Implementación y Validación de Calidad **11**
5.1. Estructura del repositorio 11
5.2. Entorno de Desarrollo, Guía de instalación y Ejecución 11
5.3. Estrategias y Casos de Prueba 12
5.4. Resultados de pruebas y problemas encontrados 12
6. Declaración de uso de Inteligencia Artificial **14**
6.1. Herramientas de IA utilizadas 14
6.2. Herramientas de IA utilizadas 14
6.3. Declaración de Responsabilidad y Autoría 15
7. Conclusión **16**
8. Bibliográfia **17**

1. Introducción

1.1. Propósito del documento

El presente documento describe la arquitectura, diseño e implementación del proyecto StrangerTEC Batalla Morse, desarrollado para el curso CE-1104 Fundamentos de Sistemas Computacionales

1.2. Descripción del proyecto

StrangerTEC es un juego interactivo para dos jugadores basado en el código Morse, inspirado en la pared de luces de la serie Stranger Things. El sistema integra una maqueta física controlada por una Raspberry Pi Pico W (LEDs, buzzer y botón pulsador) con una interfaz gráfica de escritorio desarrollada en Python con la biblioteca Tkinter. La comunicación entre ambos componentes se realiza mediante una conexión HTTP sobre WiFi.

Un jugador interactúa a través del botón físico de la maqueta, mientras que el segundo jugador utiliza la barra espaciadora del teclado en la interfaz gráfica. Ambos compiten por descifrar y replicar en código Morse una palabra objetivo seleccionada aleatoriamente, obteniendo puntaje por cada carácter correcto.

1.3 Objetivos y Alcance

Objetivo principal: Desarrollar un sistema interactivo que permita a dos jugadores comunicarse y competir utilizando el código Morse, integrando hardware (Raspberry Pi Pico W, LEDs, buzzer, botón) con software (Python, Tkinter, MicroPython, HTTP/WiFi).

Objetivos específicos:

- Implementar un panel de LEDs funcional en tres filas (A–Y, B–Z, 0–9/signos) que muestre cada carácter decodificado.
- Desarrollar la lógica de codificación y decodificación del código Morse en MicroPython y Python.
- Construir una interfaz gráfica en Tkinter con pantallas de configuración, juego y resultados.
- Establecer comunicación WiFi entre la Raspberry Pi Pico W y la PC mediante un servidor HTTP liviano.
- Implementar un sistema de puntaje por carácter correcto con pantalla final y comparación visual.

Alcance: El sistema implementa el Modo Local (Modo Transmisión Simple), donde ambos jugadores comparten la misma sesión de juego en una maqueta. El Modo Versus (dos maquetas comunicadas por WiFi) quedó fuera del alcance de esta entrega.

1.4 Definiciones, Acrónimos y Abreviaturas

Término	Definición
PWM	Modulación por Ancho de Pulso — controla el buzzer pasivo.
Raspberry Pi Pico W	Microcontrolador RP2040 con módulo WiFi CYW43439. Sistema empotrado del proyecto.
Tkinter	Biblioteca estándar de Python para la creación de interfaces gráficas de escritorio.
MicroPython	Implementación de Python optimizada para microcontroladores.
HTTP polling	Técnica donde la PC consulta periódicamente la Pico W via GET para obtener datos Morse.
74LS164	Registro de corrimiento serial-paralelo de 8 bits, usado en cadena daisy-chain para controlar 13 LEDs con 2 pines GPIO.
Unidad Morse	Duración mínima de un punto. Velocidad A = 0.2 s/unidad; Velocidad B = 0.3 s/unidad.

2. Descripción General del Sistema

2.1. Perspectiva del Producto

StrangerTEC es un sistema híbrido que integra hardware embebido con software de escritorio. La maqueta física actúa como servidor de eventos: captura la entrada Morse del jugador 1 mediante un botón pulsador, decodifica los símbolos en letras/números y los acumula hasta que la PC los solicite. La interfaz gráfica actúa como cliente y controlador central: selecciona las palabras, gestiona los turnos, captura la entrada del jugador 2 por teclado y calcula los puntajes finales.

2.2. Funcionalidades principales

- Pantalla de configuración: ingreso de nombres de jugadores, conexión WiFi a la Pico W (con verificación ping), y selección de velocidad Morse (Unidad A = 0.2 s, Unidad B = 0.3 s).
- Selección aleatoria de una palabra objetivo de una lista predefinida (máximo 16 caracteres).
- Visualización de la palabra objetivo en la pantalla de juego para ambos jugadores.
- Captura de entrada Morse del Jugador 1 desde la Pico W mediante HTTP polling cada 2 segundos.
- Captura de entrada Morse del Jugador 2 mediante la barra espaciadora del teclado, con temporización por software (presión corta = punto, presión larga = raya).
- Retroalimentación visual en tiempo real: muestra el símbolo actual, el carácter decodificado y el texto acumulado de cada jugador.
- Ventana flotante de referencia Morse (tabla completa A–Z, 0–9, signos), accesible sin interrumpir el juego.
- Pantalla de resultados con comparación carácter a carácter (verde = correcto, rojo = incorrecto), puntaje individual y anuncio del ganador.
- Opción de jugar otra ronda (nueva palabra aleatoria) o volver a la configuración inicial.

2.3. Restricciones del sistema, Supuestos y Dependencias

Restricciones:

- El sistema empotrado debe ser una Raspberry Pi Pico W programada en MicroPython.
- La interfaz gráfica debe desarrollarse en Python 3.10+ usando únicamente bibliotecas estándar o permitidas (Tkinter, requests).
- Las palabras objetivo tienen máximo 16 caracteres y solo pueden contener los caracteres definidos en el diccionario Morse del proyecto.
- El Modo Versus (dos maquetas comunicadas por WiFi) no fue implementado en esta

entrega.

Supuestos:

- El equipo que ejecuta la interfaz gráfica tiene Python 3.10+ instalado y la biblioteca requests disponible.
- La Raspberry Pi Pico W se conecta a la misma red WiFi que la PC y su dirección IP es estable durante la sesión.
- El usuario conoce los fundamentos básicos del código Morse o tiene acceso a la tabla de referencia provista en la aplicación.
- El usuario debe tener instalado en su dispositivo la Fuente ‘Minecraft’

Dependencias:

- Python 3.10+: tkinter (estándar), requests (pip install requests).
- MicroPython 1.22+: machine (Pin, PWM), time, network, socket.

Hardware: Raspberry Pi Pico W, 13 LEDs, 2× 74LS164, buzzer pasivo, botón pulsador.

3. Requisitos del Sistema

3.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	El sistema selecciona aleatoriamente una palabra de la lista y la muestra en pantalla.	Alta	La palabra mostrada corresponde a una de las entradas de PALABRAS.
RF-002	El panel de LEDs en la Pico W muestra el carácter Morse decodificado correctamente.	Alta	Cada letra/número enciende el LED correspondiente en la fila correcta.
RF-003	El buzzer suena durante toda la presión del botón como retroalimentación auditiva.	Media	El buzzer emite sonido desde que se presiona hasta que se suelta el botón.
RF-004	El botón pulsador detecta punto (presión corta) y raya (presión larga).	Media	Duración < 300 ms = punto; duración >= 300 ms = raya.
RF-005	El sistema detecta pausas entre símbolos (1 U) y entre letras (3 U) en la Pico W.	Media	Pausa > 1000 ms tras última pulsación confirma la letra y limpia el buffer.
RF-006	La PC obtiene el morse acumulado de la Pico W mediante polling HTTP	Alta	GET /GET_MORSE devuelve la cadena Morse y limpia el buffer en la Pico W.
RF-007	El sistema compara la entrada Morse del jugador con la palabra objetivo y asigna puntaje.	Alta	El puntaje refleja exactamente el número de caracteres coincidentes posición a posición.
RF-008	La pantalla final muestra comparación carácter a carácter y el ganador.	Alta	Cada carácter correcto se muestra en verde y el incorrecto en rojo.
RF-009	La interfaz valida que se ingresen los nombres de	Media	Si algún campo está vacío, se muestra un mensaje de error

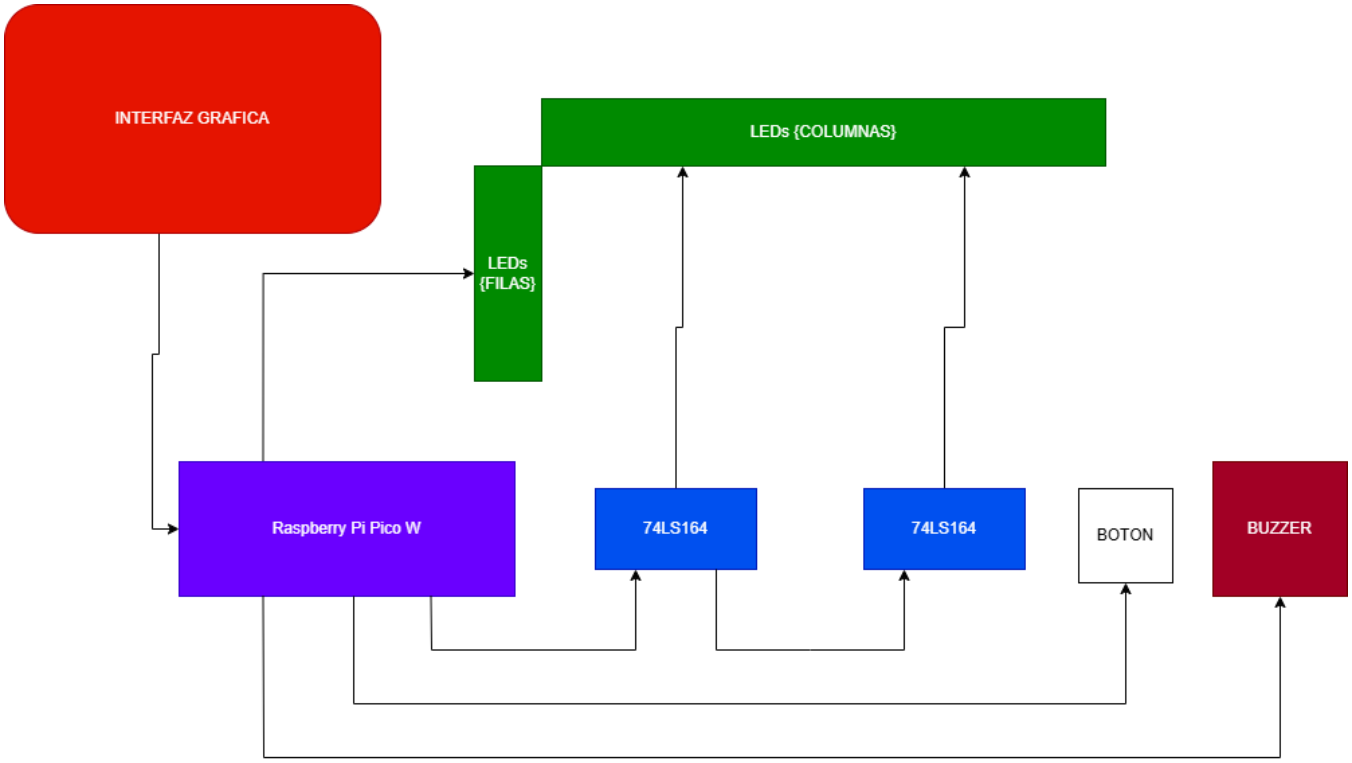
	ambos jugadores antes de iniciar.		y no avanza.

3.2. Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Rendimiento	El sistema debe responder a la presión del botón sin retardo perceptible.	Latencia de detección < 50 ms (ciclo de polling de 10 ms en la Pico W).
RNF-002	Usabilidad	La interfaz debe ser comprensible para usuarios sin experiencia previa en Morse.	El jugador puede completar una ronda sin instrucciones adicionales tras una explicación breve.
RNF-003	Confiabilidad	El sistema debe mantener el estado del juego sin pérdida de datos durante una partida.	No se pierden puntos ni se reinicia de forma inesperada durante una sesión.
RNF-004	Mantenibilidad	El código debe estar comentado para que cualquier integrante pueda orientarse.	Cada función o bloque lógico tiene comentarios descriptivos en español.
RNF-005	Portabilidad	La interfaz gráfica debe ejecutarse en Windows 10+ con Python 3.10+.	El sistema se ejecuta correctamente sin modificaciones adicionales en el SO objetivo.

4. Diseño de la Solución

4.1. Visión General de la Arquitectura



4.2. Descripción de Componentes

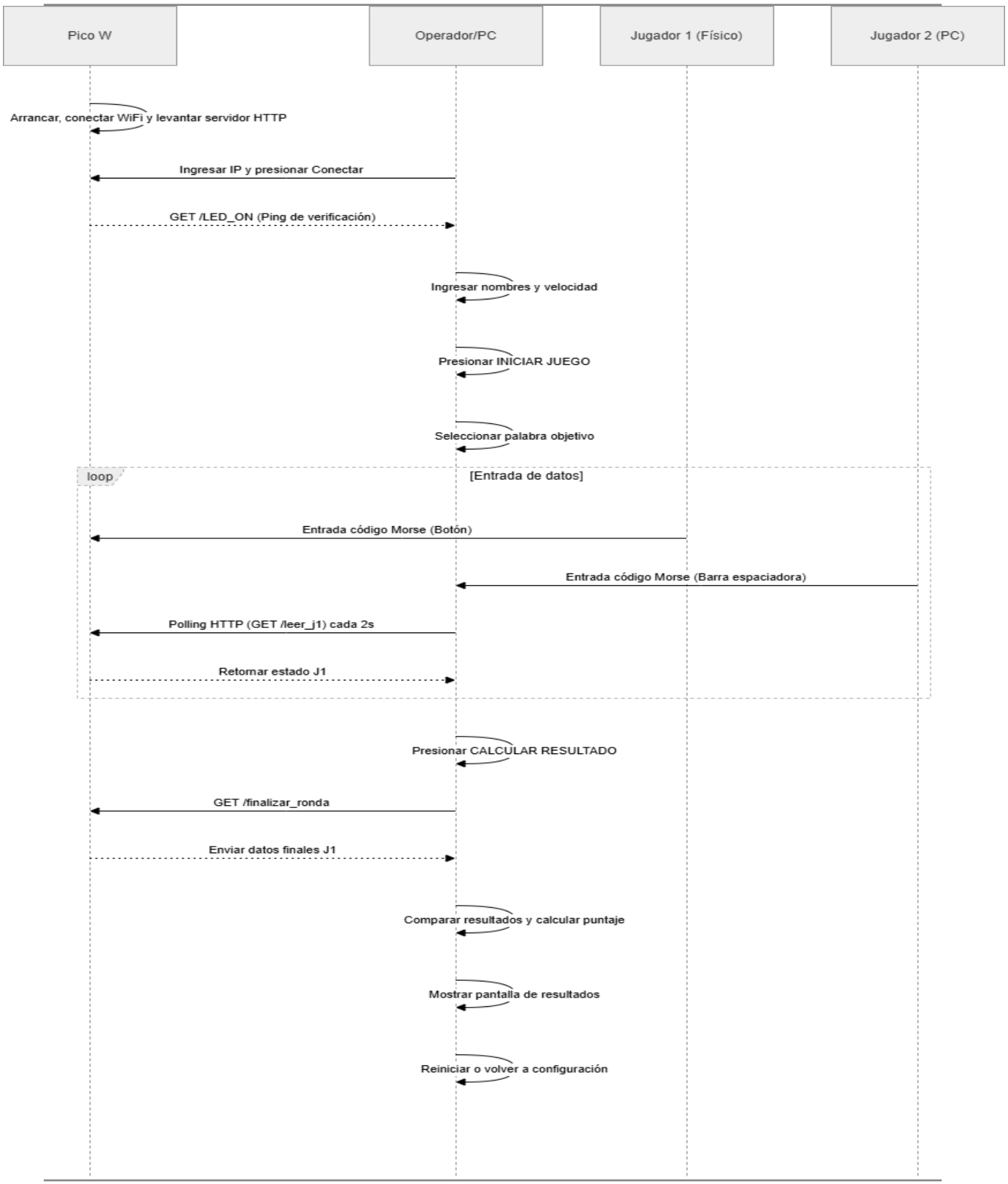
Componente	Responsabilidad	Tecnología	Dependencias
TKINTER.py	Módulo principal de la interfaz PC. Gestiona las 3 pantallas (Configuración, Juego, Resultado), la captura Morse por teclado (SPACE), el polling HTTP a la Pico W, la decodificación Morse, el cálculo de puntajes y la lógica de pantallas.	Python 3.10+ / Tkinter, requests	Pico W (HTTP GET sobre WiFi)
MICROPYTHON.py	Firmware completo de la Pico	MicroPython	Hardware: LEDs, buzzer,

(Pico W)	W. Inicializa GPIO (botón pin 16, LED pin 15, buzzer pin 18 PWM, CLK pin 27, DATA pin 13, filas LEDs pins 0/1/28). Mantiene un servidor HTTP no bloqueante en puerto 80. Detecta punto/raya por duración de presión, decodifica Morse y enciende el LED correspondiente.	1.22+ / network, socket, machine	botón, 74LS164

4.3. Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Lenguaje PC	Python	3.10+	Lógica del juego e interfaz gráfica
Framework GUI	Tkinter	Estándar	Interfaz gráfica de escritorio (sin dependencias externas)
Biblioteca HTTP	requests	2.x	Peticiones GET a la Pico W para obtener morse acumulado
Lenguaje empotrado	MicroPython	1.22+	Firmware de la Raspberry Pi Pico W
Hardware principal	Raspberry Pi Pico W	RP2040-W	Microcontrolador — servidor HTTP, control GPIO
IDE empotrado	Thonny	4.1+	Desarrollo y carga de firmware MicroPython en la Pico W

4.4. Diagrama de Flujo y Secuencia


Pico W
Operador/PC
Jugador 1 (Físico)
Jugador 2 (PC)
Arrancar, conectar WiFi y levantar servidor HTTP
Ingresar IP y presionar Conectar
GET /LED_ON (Ping de verificación)
Ingresar nombres y velocidad
Presionar INICIAR JUEGO
Seleccionar palabra objetivo
loop
[Entrada de datos]
Entrada código Morse (Botón)
Entrada código Morse (Barra espaciadora)
Polling HTTP (GET /leer_j1) cada 2s
Retornar estado J1
Presionar CALCULAR RESULTADO
GET /finalizar_ronda
Enviar datos finales J1
Comparar resultados y calcular puntaje
Mostrar pantalla de resultados
Reiniciar o volver a configuración

4.5. Decisiones de Diseño y Justificación

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Interfaz PC	PyQt, wxPython	Tkinter	Biblioteca estándar de Python, sin dependencias externas adicionales y suficiente para los requerimientos del proyecto.
Control de LEDs	Control directo pin-a-pin, Registro de corrimiento	74LS164 (daisy-chain ×2)	Permite controlar 13 LEDs con solo 2 pines del microcontrolador, liberando pines para otros componentes.
Comunicación PC-Pico	WiFi HTTP, Bluetooth, USB serial	WiFi HTTP (con USB como alternativa)	WiFi permite mayor libertad de movimiento de la maqueta. USB sirve como respaldo para desarrollo.
Concurrencia Pico W	_thread, uasyncio, polling no bloqueante	Socket no bloqueante + ciclo de polling 10 ms	Evita la complejidad de _thread y uasyncio en MicroPython. El ciclo principal alterna HTTP y lectura de botón sin bloqueo.
Polling en PC	threading, asyncio, after() de Tkinter	threading.Thread daemon	Permite ejecutar el polling HTTP sin bloquear el event loop de Tkinter. El uso de after(0,...) garantiza actualizaciones seguras de widgets desde el hilo secundario.
Almacenamiento de palabras	Archivo externo, base de datos, lista en código	Lista en código Python (PALABRAS)	Suficiente para 10 palabras. Modificable fácilmente entre partidas sin infraestructura adicional.

5. Implementación y Validación de Calidad

5.1. Estructura del repositorio

```
/StrangerTEC
  ├── MICROPYTHON.py   → Firmware MicroPython para la Raspberry Pi Pico W
  ├── TKINTER-GUI.py   → Interfaz gráfica Python/Tkinter para la PC
  └── Documentacion_Tecnica.pdf  → Este documento
```

5.2. Entorno de Desarrollo, Guía de instalación y Ejecución

Entorno:

- Sistema Operativo: Windows 10+ / macOS 12+ / Ubuntu 20.04+
- Lenguaje / Runtime: Python 3.10+ (interfaz PC), MicroPython 1.22+ (Raspberry Pi Pico W)

Guía de instalación y ejecución:

1. **Clonar el repositorio:**

   git clone https://github.com/cris]/[repositorio].git

2. **Instalar dependencias en la PC:**

   pip install requests

3. **Cargar el firmware en la Pico W:**

   Abrir Thonny → conectar la Pico W por USB → abrir MICROPYTHON.py → presionar Ejecutar (F5). Editar SSID y PASSWORD con las credenciales de la red WiFi local.

4. **Ejecutar la interfaz gráfica:**

   python TKINTER.py

5. **Ingresar la IP impresa por la Pico W en la consola de Thonny, presionar Conectar y luego INICIAR JUEGO.**

5.3. Estrategias y Casos de Prueba

Estrategias de prueba:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Unitaria	Verificación de funciones individuales de decodificación Morse.	print() de prueba en consola de Thonny / Python REPL	Funciones Decodificar_simbolo() y Decodificar_cadena()
Funcional	Validación del comportamiento del sistema desde la perspectiva del usuario final.	Pruebas manuales con la maqueta física	Casos de uso principales del juego
Rendimiento	Medición del comportamiento bajo condiciones normales de juego.	Cronómetro manual y observación visual	Latencia de detección de botón y tiempo de respuesta HTTP

Casos de prueba:

ID	Descripción	Pasos	Resultado Esperado	Resultado	Estado
CP-001	Decodificación Morse correcta	Llamar Decodificar_cadena('.-. ---')	Retorna 'AEO'	'AEO'	✅ Pasó
CP-002	Iniciar juego sin nombres	No llenar campos de nombres y presionar INICIAR JUEGO.	El sistema muestra mensaje de error y no avanza.	Error mostrado	✅ Pasó
CP-003	Código Morse inválido (símbolo desconocido)	Ingresar secuencia Morse sin valor en el diccionario.	Retorna cadena vacía '' (no agrega carácter al texto).	'' retornado	✅ Pasó
CP-004	Conexión WiFi fallida (IP incorrecta)	Ingresar IP incorrecta y presionar Conectar.	Muestra 'No se pudo conectar' y PICO_IP queda en None.	Mensaje correcto	✅ Pasó

CP-005	Cálculo de puntaje	Objetivo='SI', respuesta='NO'. Calcular_puntos()	Puntaje = 0 (ningún carácter coincide en posición).	0 retornado	☑ Pasó

5.4. Resultados de pruebas y problemas encontrados

Resultados:

- Casos de prueba exitosos: todas las pruebas unitarias de decodificación Morse funcionaron correctamente en Python REPL y en la Pico W.
- El juego puede ejecutarse de forma fluida en una sesión de múltiples rondas sin pérdida de datos.
- El polling HTTP introduce una latencia máxima de 2 segundos para recibir la entrada del jugador 1, lo cual es aceptable para el ritmo de juego.

Problemas encontrados y solucionados:

- Conflicto entre servidor HTTP y lectura de botón: al usar el socket en modo bloqueante, la espera de peticiones HTTP impedía detectar las pulsaciones del botón físico. Solución: configurar el socket en modo no bloqueante (srv.setblocking(False)) y manejar la excepción OSError cuando no hay conexiones entrantes.
- Conexion insatisfactoria entre internet movil: al intentar conectar el programa a traves de compartir datos con el dispositivo movil era insatisfactorio, al ser un cellular marca Apple en especifico un Iphone 13, este comparte internet a una frecuencia de 5GHz, lo cual es incompatible con la raspberry. Solucion: Cambiar la configuracion de el modo de compartir datos moviles
- Actualización de widgets de Tkinter desde hilo secundario: las actualizaciones directas de labels desde el hilo de polling causaban errores esporádicos. Solución: usar after(0, callback) para encolar las actualizaciones en el event loop principal de Tkinter.

6. Declaración de uso de Inteligencia Artificial y herramientas

6.1. Herramientas de IA utilizadas

Herramienta de IA	Tipo	Propósito
Claude AI	LLM	Asistencia en partes complejas del código (servidor HTTP no bloqueante, manejo de _thread, lógica de temporización Morse). Ayuda al entender el Proyecto de forma elemental, solucion de errores complejos, parte de la redaccion tecnica de este documento,
ChatGPT	LLM	Consultas tecnicas de funcionamiento de components, programas, bibliotecias, tecnicas y demas
Gemini	LLM	Ideas generales para iniciacion del Proyecto y su entendimiento general

6.2. Descripción del Uso de Inteligencia Artificial

- Claude: Generación y asistencia en código para partes técnicas complejas, como la correcta configuración del socket no bloqueante en MicroPython, la sincronización entre el hilo de polling y el event loop de Tkinter, y la lógica de temporización Morse. Solucion de errores de alta complejidad y redacción.
- ChatGPT: Consultas tecnicas de funcionamiento de components, programas, bibliotecias, tecnicas y demas
- Gemini: Ideas generales para iniciacion del Proyecto y su entendimiento general

6.3. Declaración de Responsabilidad y Autoría

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto "[Nombre del Proyecto]", declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

7. Conclusión

El proyecto StrangerTEC logró integrar exitosamente hardware embebido y software de escritorio en un sistema funcional de juego interactivo basado en el código Morse. Se cumplieron los objetivos principales: el panel de LEDs responde correctamente a las entradas del jugador, la decodificación Morse opera en tiempo real tanto en la Pico W como en la PC, y la interfaz gráfica gestiona el flujo completo del juego de forma estable entre ambos jugadores.

El desarrollo implicó superar obstáculos técnicos significativos, principalmente en la integración entre el servidor HTTP y la lectura física del botón en un microcontrolador con recursos limitados. La solución con socket no bloqueante y ciclo de polling de 10 ms resultó robusta y suficiente para los requisitos del proyecto. En el lado del hardware, el diagnóstico y reemplazo de un registro de corrimiento defectuoso reforzó la comprensión práctica del funcionamiento de circuitos digitales reales.

A nivel de software en la PC, el uso de un hilo demonio separado para el polling HTTP y el mecanismo after(0, callback) de Tkinter para la actualización segura de la interfaz resolvieron el problema de concurrencia entre el hilo de red y el event loop de la interfaz gráfica. La arquitectura cliente-servidor sobre HTTP demostró ser práctica y suficiente para la comunicación entre la Pico W y la interfaz, sin necesidad de dependencias externas complejas.

Como área de mejora identificada, el proyecto se beneficiaría de una planificación más temprana de las pruebas físicas con el hardware completo, ya que varios problemas de circuito surgieron tarde en el desarrollo. A futuro, implementar el Modo Versus entre dos maquetas comunicadas por WiFi representaría una expansión natural y viable sobre la base ya construida.

8. Bibliográfia

[1] Raspberry Pi Foundation, Raspberry Pi Pico W Datasheet, 2022. [En línea]. Disponible en: https://www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico.html

[2] MicroPython Project, MicroPython Documentation, 2024. [En línea]. Disponible en: https://micropython.org/

[3] Python Software Foundation, tkinter — Python interface to Tcl/Tk, Python 3.10 Documentation, 2024. [En línea]. Disponible en: https://docs.python.org/3/library/tkinter.html

[4] Python Software Foundation, socket — Low-level networking interface, Python 3 Documentation. [En línea]. Disponible en: https://docs.python.org/3/library/socket.html

[5] Python Software Foundation, threading — Thread-based parallelism, Python 3 Documentation. [En línea]. Disponible en: https://docs.python.org/3/library/threading.html

[6] Texas Instruments, SN74LS164 8-bit Serial-in, Parallel-out Shift Register Datasheet, 2015. [En línea]. Disponible en: https://www.ti.com/lit/ds/symlink/sn74ls164.pdf

[7] MorseCW, Alfabeto Morse y espaciado. [En línea]. Disponible en: https://morsecw.com/alfabeto.html#espaciado

[8] Instructables, The 74HC164 Shift Register and Your Arduino, 2020. [En línea]. Disponible en: https://www.instructables.com/The-74HC164-Shift-Register-and-your-Arduino/

[9] A. Vail y S. Morse, International Morse Code, ITU-R M.1677-1, International Telecommunication Union, Geneva, 2009.

[10] Python Requests Library, Requests: HTTP for Humans, 2024. [En línea]. Disponible en: https://requests.readthedocs.io/